



PYTHON ML & AI ECOSYSTEM

Python Libraries Every AI Learner Should Know



AI Coach John



PROITBRIDGE
SCHOOL FOR BETTER FUTURE

FOLLOW

Repost

Every Category You Need to Know, **At a Glance**

1 Data Handling & Utilities 

2 Deep Learning Frameworks 

3 Model Access & NLP 

4 LLM Orchestration 

5 Prompt Programming 

6 Multi-Agent Frameworks 

7 Vector DBs & Search 

8 Structured Output 

9 Evaluation 

10 Observability & Tracing 

11 Experiment Tracking 

12 Serving & Inference 

13 Fine-Tuning & Training 



Data Handling & Utilities

The libraries you'll use in almost every script

numpy

Fast numerical arrays and math operations, the foundation nearly every other library builds on.

```
import numpy as np
vector = np.array([0.1, 0.5, 0.9])
similarity = np.dot(vector, vector) / (np.linalg.norm(vector) ** 2)
```

pandas

Load, clean, filter, and analyze tabular data (CSV, Excel, SQL results).

```
import pandas as pd
df = pd.read_csv("data.csv")
df_filtered = df[df["score"] > 0.8]
```

requests

Make HTTP calls to APIs (LLM providers, data sources, webhooks).

```
import requests
response = requests.get("https://api.example.com/data")
data = response.json()
```





Deep Learning Frameworks

The engines that run the models underneath everything

torch (PyTorch)

The dominant deep learning framework; almost every open-source model (Llama, Mistral, BERT) is built on it.

```
import torch
device = "cuda" if torch.cuda.is_available() else "cpu"
tensor = torch.tensor([1.0, 2.0, 3.0]).to(device)
```

tensorflow

Google's deep learning framework; less common in new LLM work now, but still widely used in production ML and some research codebases.



When you'll actually touch these directly: Mostly when fine-tuning or running raw models. Most day-to-day LLM app work happens through higher-level libraries, not PyTorch/TF directly.





Model Access & NLP

Downloading and running actual models

transformers (HuggingFace)

The standard library for loading open-source models (LLMs, embedding models, classifiers) with a few lines of code.

```
from transformers import pipeline
classifier = pipeline("sentiment-analysis")
classifier("This library is incredibly useful.")
```

sentence-transformers

Built on top of **transformers**, specialized for generating embeddings (used everywhere in RAG and semantic search).

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("BAAI/bge-small-en-v1.5")
embedding = model.encode("What is retrieval-augmented generation?")
```



LLM Orchestration

Chaining prompts, tools, and logic together

langchain

The most widely adopted framework for building LLM apps: chains, tools, memory, retrievers, document loaders.

```
from langchain_community.document_loaders import PyPDFLoader
docs = PyPDFLoader("file.pdf").load()
```

langgraph

Built by the LangChain team, for building LLM apps as stateful graphs with branching/looping logic (used heavily in agentic RAG).

```
from langgraph.graph import StateGraph
graph = StateGraph(dict)
```

llama-index

A LangChain alternative focused specifically on data indexing and retrieval workflows, often praised for simpler RAG setup out of the box.

```
from llama_index.core import VectorStoreIndex,
SimpleDirectoryReader
documents = SimpleDirectoryReader("data").load_data()
index = VectorStoreIndex.from_documents(documents)
```



Prompt Programming & Optimization

Beyond hand-writing prompts: the production-level shift

dspy

Instead of manually tweaking prompt text, you define what you want as a program, and DSPy automatically optimizes the prompts and few-shot examples.

```
import dspy

class AnswerQuestion(dspy.Signature):
    """Answer questions using the given context."""
    context = dspy.InputField()
    question = dspy.InputField()
    answer = dspy.OutputField()

qa = dspy.Predict(AnswerQuestion)
```

guidance

Constrains and structures LLM generation token-by-token rather than hoping the prompt works.

outlines

Forces LLM output to match a schema, regex, or grammar exactly for guaranteed structured output.



Why this category matters: Hand-tuned prompts break when you swap models or scale up. These libraries treat prompting as an engineering problem.



Multi-Agent Frameworks

When one LLM call isn't enough

crewai

Framework for defining teams of role-based agents (e.g., "researcher," "writer," "reviewer") that collaborate on a task.

```
from crewai import Agent, Task, Crew  
researcher = Agent(role="Researcher", goal="Find accurate  
information")
```

autogen (Microsoft)

Multi-agent conversation framework; agents can talk to each other, use tools, and involve a human in the loop.

openai-agents (Swarm)

Lightweight, minimal multi-agent orchestration patterns from OpenAI, good for simple handoff-style agent systems.



LangGraph vs. Multi-Agent frameworks: LangGraph gives full state control; these trade some control for faster setup of role-based teams.



Vector Databases & Search

Where your embeddings actually live

faiss

Facebook AI's library for fast local similarity search; free, runs entirely on your machine, great for prototyping.

```
import faiss
index = faiss.IndexFlatIP(384)
index.add(embeddings)
```

chromadb

Open-source vector database with a simple Python-first API; popular for local dev and small apps.

qdrant-client

Client for Qdrant, built for production filtering + search.

pinecone-client

Managed vector DB: no infra to manage, scales automatically.



Production tip: Start with FAISS/Chroma locally, migrate to Qdrant/Pinecone when you need managed scaling or multi-tenant filtering.



Structured Output & Validation

Making sure the LLM's output is actually usable

pydantic

Define data schemas in Python; used everywhere to validate that LLM output matches the expected shape and types.

```
from pydantic import BaseModel

class Answer(BaseModel):
    summary: str
    confidence: float

parsed = Answer.model_validate_json(llm_output)
```

instructor

Sits on top of `pydantic` and your LLM client, forcing the model to return validated structured output directly.

```
import instructor
client = instructor.from_openai(openai_client)
result = client.chat.completions.create(response_model=Answer,
messages=[...])
```



Why this matters: LLMs are unreliable at producing raw JSON. These libraries catch and auto-correct malformed output before it breaks your app.



LLM System Evaluation

Proving your system actually works

ragas

Purpose-built for evaluating RAG pipelines: faithfulness, answer relevancy, context precision and recall.

```
from ragas import evaluate
from ragas.metrics import faithfulness
scores = evaluate(dataset, metrics=[faithfulness])
```

deepeval

Broader LLM evaluation framework, supports unit-test-style assertions for LLM output (works well with pytest).

trulens

Evaluation + tracing combined; tracks feedback functions over your app's inputs and outputs over time.



Production tip: Pick one and wire it into CI. Evaluation shouldn't be a one-time check; run it automatically when prompts or models change.



AI Coach John



FOLLOW

 Repost

Observability & Tracing

Seeing what your pipeline actually did

langsmith

LangChain's tracing and observability platform; visualizes every step, tool call, and token cost in a run.

```
import os
os.environ["LANGCHAIN_TRACING_V2"] = "true"
```

langfuse

Open-source alternative to LangSmith; tracing, prompt management, and evaluation, self-hostable.



Why this matters: Once your pipeline has more than 2 to 3 steps (retrieval, reranking, generation), print debugging stops working. You need visual traces.



Experiment Tracking

Keeping track of what you tried and what worked

mlflow

Tracks experiments, model versions, parameters, and metrics; standard in ML, increasingly used for LLM pipelines.

```
import mlflow
with mlflow.start_run():
    mlflow.log_param("chunk_size", 500)
    mlflow.log_metric("faithfulness_score", 0.87)
```

wandb (Weights & Biases)

Similar purpose, very popular for visualizing training runs and comparing experiment results side-by-side.



Why this matters: Tuning chunk size, prompts, or models means running dozens of variations. Tracking prevents losing what actually improved things.



Serving & Inference

Actually running models at scale

vllm

High-throughput inference server for open-source LLMs; handles batching and memory far better than raw transformers.

```
python -m vllm.entrypoints.openai.api_server --model
mistralai/Mistral-7B-Instruct-v0.2
```

litellm

Unified interface to call 100+ LLM providers (OpenAI, Anthropic, local models) using the same code.

```
from litellm import completion
res = completion(model="claude-sonnet-4-6", messages=[...])
```

ollama

Run open models locally with OpenAI-compatible API.

fastapi

Standard way to wrap ML pipelines into REST APIs.





Fine-Tuning & Training

When prompting isn't enough

peft

Fine-tune large models cheaply using techniques like LoRA, without retraining the full model.

```
from peft import LoraConfig, get_peft_model
config = LoraConfig(r=8, lora_alpha=16, target_modules=["q_proj",
"v_proj"])
model = get_peft_model(base_model, config)
```

trl (Transformer Reinforcement Learning)

Fine-tune models with techniques like RLHF/DPO, used to align model behavior with human preferences.

accelerate

Handles multi-GPU distributed training boilerplate.

bitsandbytes

Enables 8-bit/4-bit training & inference on GPUs.



You Don't Need All of These On Day One

Step-by-step library learning path

Start Here (Beginner)

numpy → pandas → requests → transformers → langchain

Real RAG & Agent Apps (Intermediate)

sentence-transformers → faiss / chromadb → langgraph →
pydantic

Shipping to Production

instructor → ragas / deepeval → langsmith / langfuse →
litellm / vllm → mlflow

Custom Model Training & Fine-Tuning

peft → trl → accelerate → bitsandbytes



All 25 Libraries, One Glance

CATEGORY	LIBRARIES
Data & Utilities	numpy, pandas, requests
Deep Learning	torch, tensorflow
Model Access	transformers, sentence-transformers
Orchestration	langchain, langgraph, llama-index
Prompt Programming	dspy, guidance, outlines
Multi-Agent	crewai, autogen, openai-agents
Vector DBs	faiss, chromadb, qdrant-client, pinecone-client
Structured Output	pydantic, instructor
Evaluation	ragas, deepeval, trulens
Observability	langsmith, langfuse
Experiment Tracking	mlflow, wandb
Serving	vllm, ollama, litellm, fastapi
Fine-Tuning	peft, trl, accelerate, bitsandbytes

